파이썬 반복문

학습목표

- 파이썬 반복문에 대하여 이해할 수 있습니다.
- 파이썬 반복문의 종류와 구조를 이해하고 활용할 수 있습니다.
- 파이썬 기타 제어문의 종류와 구조를 이해하고 용도에 맞게 활용할 수 있습니다.

목차

7.1 파이썬 반복문 개요

7.2 while 문

7.3 for 문

7.4 기타 제어문

7.5 효율적인 자료구조와 반복문 활용

7.1 파이썬 반복문 개요

- 파이썬 반복문은 특정 문장을 여러 번 반복하여 실행하는 반복적 구조로서 파이썬 반복문은 조건식이 만족하는 동안 또는 범위의 횟수 동안 여러 번 반복하여 실행하는 제어문 문장
- 파이썬의 반복문의 종류에는 while 문과 for 문이 있음, while 문은 조건식의 결과가 True이면 특정 문장을 여러 번 반복 실행하다가 조건식이 False 가 되면 반복을 종료하는 반복문이고,
  for 문은 범위의 횟수 동안 특정 문장을 여러 번 반복하여 실행하는 반복문임
- 7장에서는 파이썬 반복문의 종류 while 문과 for 문에 대하여 학습하고, 기타 제어문인 break 문과 continue 문에 대하여 학습

7.1 파이썬 선택문 개요

- 파이썬 반복문 형태인 while 문과 for 문의 공통 규칙과 각각의 규칙은 다음과 같음
- 파이썬 반복문의 규칙

파이썬 반복문의 규칙

1. while 문과 for 문의 조건식 끝에는 항상 : (콜론) 이 있어야 합니다.
2. 실행 코드는 반드시 공백(스페이스바 또는 Tab)으로 들여쓰기(indent)하여 while 문과 for 문에 포함되는(종속) 코드로 작성해야 합니다.

while 문의 규칙

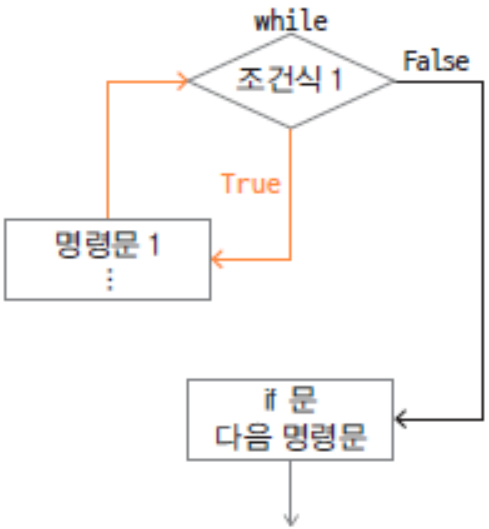
1. 선택문처럼 조건식의 결과가 True 또는 False인지에 따라 특정 문장을 반복 실행하거나 실행하지 않는 조건적 반복 실행 구조입니다.
2. while 문의 조건식들은 논리 연산이 가능한 문장, 즉 True · False 판별이 가능한 것이어야 합니다.

for 문의 규칙

1. 제어변수와 특정 범위를 활용하여 반복 실행합니다.
2. 특정 범위는 range() 함수, 리스트, 문자열 등을 이용하여 지정합니다.

7.2 while 문

(1) while 문의 기본 구조

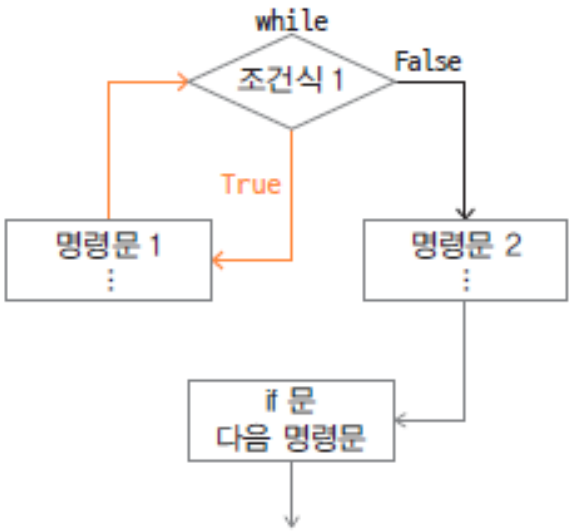
기본 문법	순서도
while 조건식: (공백) 명령문 1 ⋮ while 문 다음 명령문	 그림 7.1

설명 while 문의 조건식 결과가 True라면 명령문 1, … 을 실행하고, 이후 while 문의 조건식을 다시 확인하여 결과가 True라면 명령문 1, … 을 실행하는 과정을 반복하다가 조건식의 결과가 False가 되면 명령문 1, … 을 건너뛰고 while 문 구조를 벗어나 while 문 다음 명령문을 실행합니다.

7.2 while 문

(2) while~else 문의 기본 구조

기본 문법	순서도

while 조건식:
(공백) 명령문 1
⋮
else:
(공백) 명령문 2
⋮
while 문 다음 명령문
	
	그림 7.2

설명 while 문의 조건식 결과가 True라면 명령문 1, … 을 실행하고, 이후 while 문의 조건식을 다시 확인하여 결과가 True라면 명령문 1, … 을 실행하는 과정을 반복하다가 조건식의 결과가 False가 되면 명령문 2, … 을 실행하고 while 문 구조를 벗어나 while 문 다음 명령문을 실행합니다.

7.2 while 문

- while 문의 year<=3 조건식을 만족할 때까지 year 변수를 1씩 증가시키며 '서당개 year 년' 문장을 출력, year 변수가 4가 되면 while 문을 벗어나 '풍월을 읊습니다.' 문장을 출력하고 종료

예제 7.1 while 문을 이용한 반복 출력(1)

```
year = 1
while year <= 3 :
    print(f'서당개 {year}년')
    year += 1

print('풍월을 읊습니다.')
```

실행결과

```
서당개 1년
서당개 2년
서당개 3년
풍월을 읊습니다.
>>>
```

7.2 while 문

- 'y'를 입력하면 while 문의 result != 'y' 조건식을 만족하지 않기 때문에 while 문을 벗어나 '종료' 문장 출력 후 종료하고, 그 외의 값을 입력하면 조건식을 만족하여 '파이썬 최고!' 문장을 출력 후 입력 받는 과정을 반복 실행

예제 7.2 while 문을 이용한 반복 출력(2)

```
1  result = None
2  while result != 'y':
3      print('파이썬 최고!')
4      result = input('계속하려면 입력:(종료:y) ')
5
6  print('종료')
```

실행결과

```
파이썬 최고!
계속하려면 입력:(종료:y) hi
파이썬 최고!
계속하려면 입력:(종료:y) 1234
파이썬 최고!
 실행결과
파이썬 최고!
계속하려면 입력:(종료:y) hi
파이썬 최고!
계속하려면 입력:(종료:y) 1234
파이썬 최고!
계속하려면 입력:(종료:y) y
종료
>>>
```

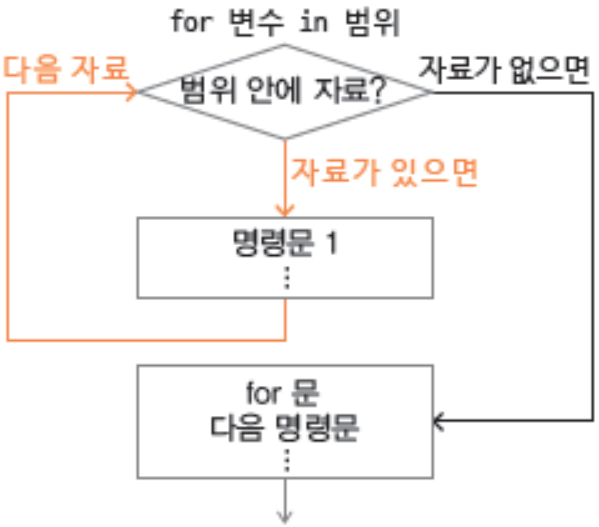
7.3 for 문

- for 문은 range() 함수, 리스트, 문자열 등을 이용하여 특정 범위를 지정하게 되는데, range() 함수를 이용하는 경우 시작 값부터 종료 값까지 증가 값 만큼씩 증가시키면서 항목의 원소를 하나씩 꺼내와 제어 변수에 저장한 뒤, for 문에 종속된 명령문들을 실행

- range() 함수는 range(0, 10, 1)과 range(0, 10)과 range(10)은 모두 0부터 9까지 1씩 증가하며 숫자를 하나씩 가져와 제어 변수에 저장 즉, 시작 값을 생략하면 0이 되고, 증가 값을 생략하면 1이 됨, 하지만 종료 값은 생략할 수 없으며, 시작 값과 증가 값은 정수형이어야 하고, 증가 값은 0이 될 수 없음

7.3 for 문

(1) for 문의 기본 구조

기본 문법	순서도
① ② ③ for 제어변수 in range(시작값, 종료값+1, 증가값) : (공백) 명령문 1 ⋮ for 문 다음 명령문	 그림 7.3
for 제어변수 in 리스트 : (공백) 명령문 1 ⋮ for 문 다음 명령문	
for 제어변수 in '문자열' : (공백) 명령문 1 ⋮ for 문 다음 명령문	

7.3 for 문

- range(5) 는 0부터 4까지 1씩 증가하면서 실행하는데, 처음에는 0의 값을 가져와 제어변수 i 에 저장 후, '*' 문자를 i+1 (0+1) 만큼 반복 출력

예제 7.3 range() 함수를 이용한 반복 출력

```
for a in range(5) :
   print('*' * (a+1))
```

실행결과

```
*
**
***
****
*****
>>>
```

7.3 for 문

- odd 리스트에서 항목을 하나씩 가져와 반복 실행하는 예제
  - odd 리스트의 1을 가져와 제어변수 i 에 저장 후, i+1 (1+1)을 출력
  - 이후 odd 리스트의 다음 항목 3을 가져와 제어변수 i 에 저장 후 i+1 (3+1) 출력
  - 이 과정을 반복하게 되는데 마지막 항목 9를 가져와 제어변수 i 에 저장 후 i+1 (9+1)을 출력하고 for 문 종료

예제 7.4 리스트를 이용한 반복 출력

```
odd = [1, 3, 5, 7, 9]
for a in odd :
    print(a+1, end=' ')
```

실행결과

```
2 4 6 8 10
>>>
```

7.3 for 문

- 딕셔너리에서 키를 하나씩 가져와 반복 실행하는 예제
  - for 문에서 딕셔너리의 항목을 가져올 때는 키만 가져옴

예제 7.5 딕셔너리를 이용한 반복 출력

```
1  book = {'철학':100, '기술과학':500, '문학':800}
2  for a in book :
3      print(a, end=' ')
```

실행결과

```
철학 기술과학 문학
>>>
```

7.3 for 문

- 'Python' 문자열에서 문자를 하나씩 가져와 반복 실행하는 예제
  - 처음에는 'Python' 문자열의 문자 'P'를 가져와 제어변수 i 에 저장 후, i ('P') 출력
  - 이후 'Python' 문자열의 다음 문자 'y'를 가져와 제어변수 i 에 저장 후 i ('y') 출력
  - 이 과정을 반복하게 되는데 마지막 문자 'n'을 가져와 제어변수 i 에 저장 후 i (n')을 출력하고 for 문 종료

예제 7.6 문자열을 이용한 반복 출력

```
1  for a in 'Python' :
2     print(a, end=' ')
```

실행결과

```
P y t h o n
>>>
```

7.3 for 문

- 단을 입력 받아 구구단을 출력하는 예제
  - 위의 예제는 for 문에서 range() 함수를 이용하여 1부터 9까지 반복하면서 구구단을 출력
  - 아래 예제는 while 문을 이용하여 조건식 i <= 9를 만족할 때까지 반복하면서 구구단을 출력

예제 7.7 단을 입력받아 구구단 출력

```
num = int(input('단 입력: '))
for a in range(1, 10):
    print(f'{num} × {a} = {num*a}')
```

```
a = 1
num = int(input('단 입력: '))
while a <= 9 :
    print(f'{num} × {a} = {num*a}')
    a += 1
```

실행결과

```
단 입력: 7
7 × 1 = 7
7 × 2 = 14
7 × 3 = 21
7 × 4 = 28
7 × 5 = 35
7 × 6 = 42
7 × 7 = 49
7 × 8 = 56
7 × 9 = 63
>>>
```

7.4 기타 제어문

(1) break 문을 이용한 강제 종료

- for 문이나 while 문 안의 명령문1, … 을 실행하다가 탈출 조건식 결과에 따라 True 이면 break 문을 만나게 되어 반복 실행 구간의 명령문2, ··를 실행하지 않고 for 문이나 while 문을 벗어나 다음 명령문을 실행
- 탈출 조건식의 결과가 False 이면 for 문이나 while 문의 반복 실행 구간을 계속 실행

순서도

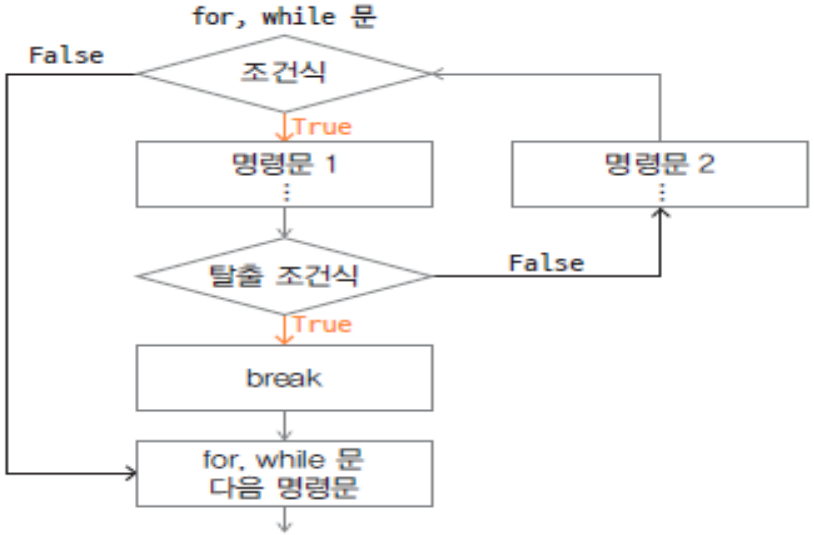


그림 7.4

7.4 기타 제어문

(2) continue 문으로 반복 구간 건너 띄기

- for 문이나 while 문 안의 명령문1, … 을 실행하다가 탈출 조건식 결과에 따라 True 이면 continue 문을 만나게 되어 반복 실행 구간의 명령문2, … 를 실행하지 않고 for 문이나 while 문의 처음으로 이동
- 탈출 조건식의 결과가 False 이면 for 문이나 while 문의 반복 실행 구간을 계속 실행

순서도

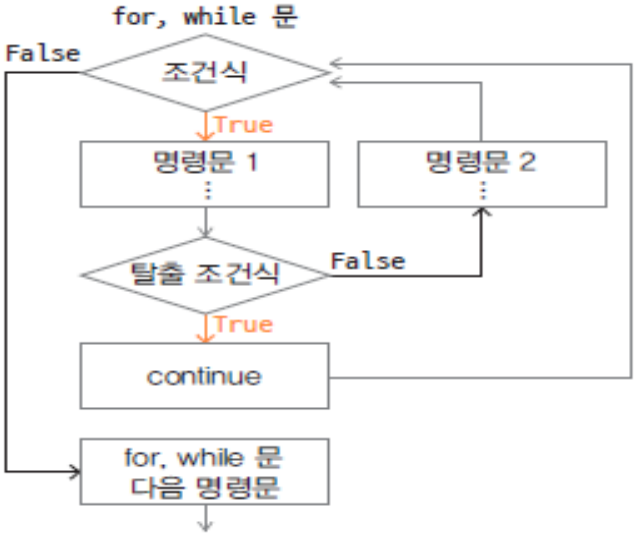


그림 7.4

7.4 기타 제어문

- while True 문에 따라 무한 반복을 실행하게 되는데, 내부에 탈출 조건식 num == 0 을 만족하게 되면 beak 문을 만나게 되어 while 문을 강제 종료

예제 7.8 무한루프 while 문 안에서 break를 이용한 강제 종료

```
while True:
    num = int(input('번호 입력(종료 0): '))
    if num == 0 :
        break
    print('while 무한루프로 반복 중')
```

실행결과

```
번호 입력(종료 0): 1
while 무한루프로 반복 중
번호 입력(종료 0): 3
while 무한루프로 반복 중
번호 입력(종료 0): 0
>>>
```

7.4 기타 제어문

- 1~30 사이의 정수 중에서 7의 배수를 출력하는 예제
  - 위의 예제는 for 문안에 if 문을 이용하여 7 로 나눈 나머지가 0 이면 '7의 배수' 문장을 출력
  - 아래 예제는 for 문안에 if 문을 이용하여 7 로 나눈 나머지가 0 이 아니면 continue 문을 만나게 되어 '7의 배수' 문장을 출력하지 않고 for 문의 처음으로 이동

예제 7.9 1~30 사이의 정수 중에서 7의 배수 출력

```
for a in range(1,31) :
    if a % 7 == 0 :
        print(f'7의 배수: {a}')
```

```
for a in range(1,31) :
    if a % 7 != 0 :
        continue
    print(f'7의 배수: {a}')
```

실행결과

```
7의 배수: 7
7의 배수: 14
7의 배수: 21
7의 배수: 28
>>>
```

7.4 기타 제어문

- pass 문은 아무것도 실행하지 않는 문장이다
  - 반복문이나 임시 함수 작성 시 기본 틀만 표현하려고 내부에 당장 아무것도 작성하지 않으면 오류가 발생
  - 이럴 때 임시적으로 pass 문을 작성해 놓으면 오류없이 기본 틀을 갖출 수 있음, 이후 실제 코드를 작성할 때 pass 문을 삭제

- pass 문의 사용 형태

예제 7.10 pass 문 사용 형태

```
while True:
    pass
```

```
for a in range(10):
    pass
```

7.5 효율적인 자료구조와 반복문 활용

- 리스트에 저장된 점수를 차례대로 읽어 와서 리스트 저장된 번호 순으로 '합격' 또는 '불합격'을 출력하고 합격한 학생 수를 출력 (합격 기준은 점수가 80점 이상)

예제 7.11 합격/불합격 판단하기

```
1  score=[60,80,90,70,95]
2  cnt=0
3  print(score)
4  for i in range(len(score)):
5      if score[i]>=80:
6          result='합격'
7          cnt += 1
8      else:
9          result='불합격'
10     print(f'{i+1}번 학생은 {result}입니다.')
11
12 print(f'합격한 학생 수: {cnt}')
```

실행결과

```
[60, 80, 90, 70, 95]
1번 학생은 불합격입니다.
2번 학생은 합격입니다.
3번 학생은 합격입니다.
4번 학생은 불합격입니다.
5번 학생은 합격입니다.
합격한 학생 수: 3
>>>
```

7.5 효율적인 자료구조와 반복문 활용

- 사용자로부터 5개의 점수를 입력받아 리스트에 저장하고 리스트에 저장된 점수에 대한 합계, 평균, 최대값, 최소값을 출력

예제 7.12 평균, 최대값, 최소값 구하기

```
hap=0
score=[0]*5        # 리스트 초기화

for i in range(len(score)):
    score[i]=int(input(f'{i+
    hap += score[i]

# 평균, 최대값, 최소값 구하기
avg=hap/len(score)
max_score=max(score)
min_score=min(score)

print('='*20)
print(f'합  계 => {hap}')
print(f'평  균 => {avg:.2f}')
print(f'최대값 => {max_score}')
print(f'최소값 => {min_score}')
print(f'='*20)
```

실행결과

```
1번째 점수:85
2번째 점수:45
3번째 점수:96
4번째 점수:55
5번째 점수:75
```

실행결과

```
1번째 점수:85
2번째 점수:45
3번째 점수:96
4번째 점수:55
5번째 점수:75
====================
합  계 => 356
평  균 => 71.20
최대값 => 96
최소값 => 45
====================
>>>
```

7.5 효율적인 자료구조와 반복문 활용

- 딕셔너리에 학생 식당 메뉴(키)와 가격(값)을 저장하고 전체 정보를 출력한 후 메뉴에 대한 가격을 검색해서 출력해 주고, 사용자가 'q'를 입력하면 종료

키(key)	값(value)
라면	3000
떡볶이	4000
김밥	2000
햄버거	5000

예제 7.13 메뉴와 가격 정보 출력하기

```
menu={}

menu['라면']=3000
menu['떡볶이']=4000
menu['김밥']=2000
menu['햄버거']=5000

for m in menu:
    print(f'{m}-{menu[m]}')

while True:
    s=input('search menu : ')
    if s=='q': break
    print(menu.get(s,'Not Found'))
```

실행결과

```
라면-3000
떡볶이-4000
김밥-2000
햄버거-5000
search menu : 김밥
2000
search menu : 라면
3000
search menu : 참치마요
Not Found
search menu : q
>>>
```

7.5 효율적인 자료구조와 반복문 활용

- 학생들의 이름과 점수를 저장한 딕셔너리를 만들고, 반복문을 사용하여 각 학생의 이름과 점수를 출력하고 학생들의 평균 점수를 계산한 후 출력

예제 7.14 학생 정보 딕셔너리와 반복문 활용

```
# 학생 정보 딕셔너리 생성
students = {
    '대한이': 85,
    '민국이': 72,
    '행복이': 91,
    '사랑이': 68,
    '기쁨이': 97
}

# 모든 학생의 이름과 점수 출력하기
for student, score in students.items():
    print(f'{student}: {score}')

# 학생들의 평균 점수 계산하기
total_score = 0
for score in students.values():
    total_score += score

average_score = total_score / len(students)
print(f'전체 학생들의 평균 점수: {average_score:.2f}')
```

실행결과

```
대한이: 85
민국이: 72
행복이: 91
사랑이: 68
기쁨이: 97
전체 학생들의 평균 점수: 82.60
>>>
```

수고하셨습니다.